```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import
confusion_matrix,accuracy_score,precision_score,recall_score,f1_score
import seaborn as sns

df=pd.read_csv("C:/Users/User/Downloads/diabetes.csv")

df.head()
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI
0	6	148	72	35	0	33.6
1	1	85	66	29	0	26.6
2	8	183	64	0	0	23.3
3	1	89	66	23	94	28.1
4	0	137	40	35	168	43.1

	Pedigree	Age	Outcome
0	0.627	50	1
1	0.351	31	0
2	0.672	32	1
3	0.167	21	0
4	2.288	33	1

```
df.isnull().sum()
```

```
Pregnancies      0
Glucose          0
BloodPressure    0
SkinThickness    0
Insulin          0
BMI              0
Pedigree         0
Age              0
Outcome          0
dtype: int64
```

```
df.columns
```

```
Index(['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness',
'Insulin',
       'BMI', 'Pedigree', 'Age', 'Outcome'],
      dtype='object')
```

```
cols_to_replace=['Glucose','BloodPressure','SkinThickness','Insulin','
BMI']
for column in cols_to_replace:
    data[column].replace(0,np.nan,inplace=True)

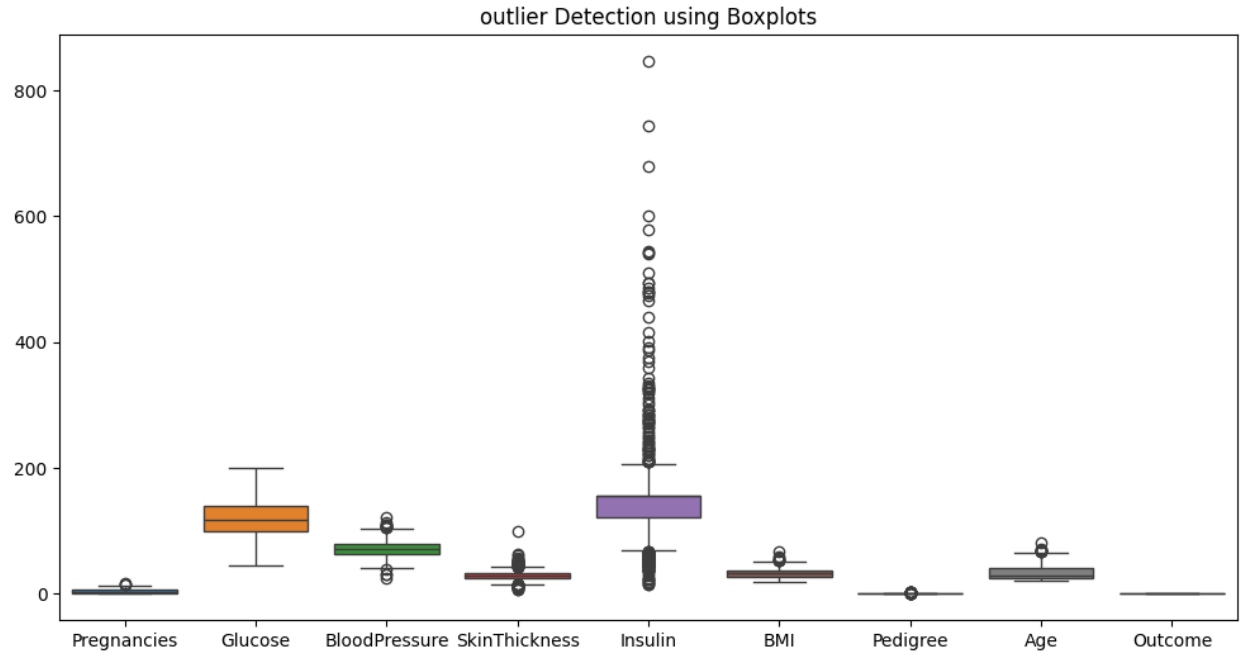
data[column].fillna(round(data[column].mean(skipna=True)),inplace=True
)

X=data.iloc[:,:8]
Y=data['Outcome']

X_train,X_test,Y_train,Y_test=train_test_split(X,Y,test_size=0.2,rando
m_state=0)

import matplotlib.pyplot as plt
plt.figure(figsize=(12,6))
sns.boxplot(data)
plt.title("outlier Detection using Boxplots")
plt.show()
Q1=data.quantile(0.25)
Q3=data.quantile(0.75)
IQR=Q3-Q1

outliers=((data<(Q1-1.5*IQR)) | (data>(Q3 + 1.5 *IQR))).sum()
print("\n Number of outliers per Feature: \n",outliers)
```




```
 Number of outliers per Feature:
 Pregnancies        4
```

```
Glucose              0
BloodPressure       14
SkinThickness       87
Insulin            159
BMI                  8
Pedigree            29
Age                  9
Outcome              0
dtype: int64
```

```
knn=KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train ,Y_train)
```

```
KNeighborsClassifier()
```

```
knn_pred=knn.predict(X_test)
```

```
cm=confusion_matrix(Y_test,knn_pred)
accuracy=accuracy_score(Y_test,knn_pred)
error_rate=1-accuracy
precision=precision_score(Y_test,knn_pred)
recall=recall_score(Y_test,knn_pred)
f1=f1_score(Y_test,knn_pred)
```

```
print("Confusion Matrix:\n",cm)
print("Accuracy Score:",accuracy)
print("Error Rate:", error_rate)
print("Precision Score :", precision)
print("Recall Score:",recall)
print("F1 Score:",f1)
```

```
Confusion Matrix:
 [[88 19]
 [19 28]]
Accuracy Score: 0.7532467532467533
Error Rate: 0.24675324675324672
Precision Score : 0.5957446808510638
Recall Score: 0.5957446808510638
F1 Score: 0.5957446808510638
```

```
accuracy_scores=[]
for k in [3, 5, 5]:
    knn=KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train, Y_train)
    knn_pred=knn.predict(X_test)
    acc=accuracy_score(Y_test,knn_pred)
    accuracy_scores.append(acc)
    print(f"k={k}->Accuracy={acc*100:.2f}%")
plt.plot([3, 5, 7], accuracy_scores, marker='o')
plt.title("KNN Accuracy vd K value")
plt.xlabel("K Value")
```

```
plt.ylabel("Accuracy")
plt.grid(True)
plt.show()

k=3->Accuracy=72.73%
k=5->Accuracy=75.32%
k=5->Accuracy=75.32%
```

